

MAIE 5102

Fundamentals of Artificial Intelligence

Lecture 1–2 Complete Notes

目录

Lecture 1: Introduction to Artificial Intelligence	12
1 What is AI? ——四种定义	12
2 Rational Agent (理性智能体)	12
3 Agent 与 Environment	13
4 Environment 分类	13
5 Search 与 Rationality 的关系	14
6 MDP ——Markov Decision Process	14
7 Bellman Equation (贝尔曼方程)	15
8 Q-learning (强化学习)	16
9 CSP (约束满足问题)	16
10 Lecture 1 核心逻辑	17
11 考试真正核心	17
Lecture 2: Linear Regression & Classification	18
12 Machine Learning Overview	18
13 Linear Regression (线性回归)	18

目录	2
14 Linear Classification (线性分类)	20
15 Regression vs Classification 对比	22
16 Lecture 2 真正考试核心	22
Lecture 3: Linear Classifier for Image Classification	23
17 Image Classification Problem	23
18 Image Classification Pipeline	23
19 Nearest Neighbor (NN)	24
20 Linear Classifier 回顾	25
21 Multi-class Linear Classifier	25
22 Training: Logistic Loss	26
23 KNN vs Linear Classifier	26
24 Non-linear Features	27
25 Decision Boundary Example	28
26 Lecture 3 核心总结	28
Lecture 4: Neural Networks and Backpropagation	29
27 From Linear to Non-linear Predictors	29
28 Neural Network Structure	29
29 Deep Neural Networks	30
30 Why Depth?	30
31 Training a 2-Layer Network	30
32 Why Backpropagation?	31
33 Backpropagation Core Idea	31
34 Simple Example	32

目录	3
35 Gradient Flow Patterns	32
36 Backprop Example with Sigmoid	33
37 Lecture 4 总结	34
Lecture 5: Unsupervised Learning	35
38 Supervised vs Unsupervised Learning	35
39 Clustering Task	35
40 Centroids	35
41 K-means Objective	36
42 Alternating Minimization	36
43 K-means Algorithm	37
44 Gradient Descent Review	37
45 Stochastic Gradient Descent	37
46 Mini-batch SGD	38
47 Generalization	38
48 Overfitting	38
49 Strategy 1: Regularization	39
50 Strategy 2: Early Stopping	39
51 Hyperparameters	39
52 Validation Set	40
53 K-Fold Cross Validation	40
54 Lecture 5 Summary	40
Lecture 5: Unsupervised Learning	42
55 Supervised vs Unsupervised Learning	42

目录	4
56 Clustering Task	42
57 Centroids	42
58 K-means Objective	43
59 Alternating Minimization	43
60 K-means Algorithm	44
61 Gradient Descent Review	44
62 Stochastic Gradient Descent	44
63 Mini-batch SGD	45
64 Generalization	45
65 Overfitting	45
66 Strategy 1: Regularization	46
67 Strategy 2: Early Stopping	46
68 Hyperparameters	46
69 Validation Set	47
70 K-Fold Cross Validation	47
71 Lecture 5 Summary	47
Lecture 6: Uninformed Search	49
72 Search Problem Definition	49
73 General Tree Search	49
74 Uninformed Search	50
75 Depth-First Search (DFS)	50
76 Breadth-First Search (BFS)	51
77 DFS vs BFS	52

目录	5
78 Iterative Deepening Search (IDS)	52
79 Uniform-Cost Search (UCS)	53
80 Lecture 6 总结	54
Lecture 7: Informed Search	55
81 Informed vs. Uninformed Search	55
82 Greedy Search	55
83 From Greedy to A*	56
84 A* Search	56
85 Is A* Optimal?	57
86 Admissible Heuristic	57
87 Optimality of A* (Tree Search)	58
88 Heuristic Design	59
89 8-Puzzle Heuristics	59
90 Heuristic Dominance	60
91 Combining Heuristics	60
92 Properties of A*	61
93 Lecture 7 总结	61
Lecture 8: Constraint Satisfaction Problems	62
94 What is Search For?	62
95 CSP Definition (形式定义)	62
96 Example: Map Coloring (地图染色)	63
97 Types of Constraints (约束类型)	63
98 Standard Search Formulation	64

目录	6
99 Naive DFS Complexity	64
100 Backtracking Search (回溯搜索)	64
101 Ordering Heuristics (排序启发式)	65
101.1 Minimum Remaining Values (MRV)	65
101.2 Least Constraining Value (LCV)	66
102 Filtering (过滤)	66
102.1 Forward Checking	66
102.2 Arc Consistency (弧一致性)	66
103 Iterative Improvement (局部搜索法)	67
103.1 Min-Conflicts	67
104 Lecture 8 Summary	67
Lecture 9: Local Search Algorithms	69
105 Introduction to Local Search	69
106 Local Search Characteristics	69
107 Hill Climbing Algorithm (爬山算法)	70
107.1 State Space Landscape (状态空间地形)	70
107.2 Hill Climbing on n-Queens	71
107.3 Escaping Local Optima	71
108 Simulated Annealing (模拟退火)	71
109 Local Beam Search (局部束搜索)	72
110 Genetic Algorithm (遗传算法)	73
110.1 Fitness Function (适应度函数)	73
110.2 Genetic Operators	73
111 Genetic Algorithm Procedure	74
112 Lecture 9 Summary	74
Lecture 10: Markov Decision Process I	76
113 From Search to MDP	76

目录	7
114 Definition of Markov Decision Process	76
115 MDP vs Search Problem	77
116 Utility and Discounting	77
117 Policy	78
118 Policy Evaluation	78
118.1 Bellman Expectation Equation	78
119 Iterative Policy Evaluation	79
120 Optimal Value	79
121 Value Iteration Algorithm	80
122 Extracting Optimal Policy	80
123 Lecture 10 Summary	80
Lecture 11: Markov Decision Process II	82
124 Roadmap	82
125 Recap of MDP	82
126 Value Iteration Review	82
127 Problems with Value Iteration	83
128 Policy Iteration	83
128.1 Policy Evaluation	83
128.2 Policy Improvement	84
128.3 Policy Iteration Algorithm	84
129 Value Iteration vs Policy Iteration	85
130 MDP Algorithms Summary	85
131 Offline vs Online Planning	86
131.1 Offline Planning	86
131.2 Online Planning	86

目录	8
132 Preview: Reinforcement Learning	86
133 Lecture 11 Summary	87
Lecture 12: Reinforcement Learning I	88
134 Introduction to Reinforcement Learning	88
134.1 From MDP to RL	88
134.2 Basic Idea of RL	88
135 Offline vs Online	89
135.1 Offline Planning (MDP)	89
135.2 Online Planning (RL)	89
136 Model-Based Reinforcement Learning	89
136.1 Key Idea	89
136.2 Step 1: Learn the Model	89
136.3 Step 2: Solve Learned MDP	90
136.4 Advantages and Disadvantages	90
137 Model-Free Reinforcement Learning	90
137.1 Direct Evaluation	90
137.2 Direct Evaluation Problems	91
138 Temporal Difference (TD) Learning	91
138.1 Big Idea	91
138.2 Sample Value	92
138.3 TD Update Rule	92
138.4 Temporal Difference	92
138.5 Why TD is Better	93
139 Problem with TD Value Learning	93
140 Lecture 12 Summary	93
Lecture 13: Reinforcement Learning II	94
141 From Value Learning to Q-Learning	94
142 Recall: Value Iteration	94
143 Q-Value Iteration	94

目录	9
144 Temporal Difference Q-Learning	95
144.1 Big Idea	95
144.2 TD Error for Q	95
145 Exploration vs Exploitation	95
146 Generalizing Across States	96
147 Feature-Based Representation	96
148 Linear Q-Function	96
149 Approximate Q-Learning	97
150 Deep Q-Network (DQN)	97
151 Training the Q-Network	97
152 Case Study: Atari Games	98
152.1 Problem Setup	98
153 Lecture 13 Summary	98
Lecture 14: Games I	99
154 Introduction to Game Problems	99
154.1 Game Playing Progress	99
155 Types of Games	99
155.1 General Games	99
155.2 Zero-Sum Games	99
156 Two-Player Zero-Sum Game Model	100
157 Game Tree	100
158 Value of a State	100
159 Adversarial Game Tree	101
160 Minimax Algorithm	101
161 Minimax Complexity	101

目录	10
162 Evaluation Function	102
163 Linear Evaluation Function	102
164 Depth-Limited Minimax	103
165 Depth-Limited Complexity	103
166 Lecture 14 Summary	103
Lecture 15: Exam Templates	104
167 Template 1: Value Iteration (MDP Calculation)	104
168 Template 2: Q-Learning Update	104
169 Template 3: A* and Admissibility	105
170 Template 4: Minimax Calculation	106
171 Template 5: CSP Filtering	106
172 Template 6: Backpropagation	107
173 Final Exam Strategy	107
174 Problem 1: BFS vs DFS vs UCS	108
175 Problem 2: A* Optimality	108
176 Problem 3: Heuristic Dominance	109
177 Problem 4: CSP Forward Checking vs Arc Consistency	109
178 Problem 5: Hill Climbing vs Simulated Annealing	110
179 Problem 6: Value Iteration Computation	110
180 Problem 7: Policy Iteration	110
181 Problem 8: Q-Learning Update	111
182 Problem 9: Backpropagation	111
183 Problem 10: Minimax and Depth-Limited Search	111

目录	11
184 Final Advice	112
Lecture 16: Exam Operating System	113
185 Part 1: Emergency Recognition Table	113
186 Part 2: Forced First Line Rule	114
187 Part 3: Anti-Stuck Protocol	115
188 Part 4: English Safety Notes	116
189 Part 5: Mechanical Exam Algorithm	116
190 Final Rule	116

Lecture 1: Introduction to Artificial Intelligence

1 What is AI? ——四种定义

AI 的四种经典定义 (Russell & Norvig):

	Human-like	Rational
Thinking	Think like people	Think rationally
Acting	Act like people	Act rationally

解释:

- Think like people: 研究人类思维机制 (认知科学)
- Act like people: 通过 Turing Test
- Think rationally: 基于逻辑推理
- Act rationally: 最大化期望性能

考试通常不考历史, 而考: “现代 AI 主要采用哪种定义?”

答案:

Act Rationally

2 Rational Agent (理性智能体)

定义:

An agent that acts to maximize expected performance.

数学表达:

$$a^* = \arg \max_a \mathbb{E}[Performance]$$

逐符号解释:

- a^* : 最优动作
- $\arg \max$: 选取使表达式最大的动作
- $\mathbb{E}$: 期望 (因为环境可能有不确定性)

考试如何体现?

- UCS 为什么 optimal?
- Bellman 方程为什么有 $\max_a$?
- Q-learning 为什么取最大 Q?

本质都来自:

maximize expected performance

3 Agent 与 Environment

Agent 通过:

- Sensors 感知环境
- Actuators 作用环境

形式化表示:

$$f : P^* \rightarrow A$$

含义:

- 输入: 所有历史感知
- 输出: 动作

Policy 表达:

$$a_t = \pi(s_t)$$

(在 MDP 假设下只依赖当前状态)

4 Environment 分类

五个维度:

- Fully observable vs Partially observable
- Deterministic vs Stochastic
- Episodic vs Sequential

- Static vs Dynamic
- Discrete vs Continuous

确定性环境:

$$s_{t+1} = f(s_t, a_t)$$

随机环境:

$$P(s_{t+1}|s_t, a_t)$$

你的 Search 样题默认是 deterministic。

因为:

每个 action 只有一个 successor。

5 Search 与 Rationality 的关系

UCS 规则:

Expand node with smallest $g(n)$

其中:

$$g(n) = \text{path cost}$$

为什么 optimal?

因为它实现:

minimize total cost

这等价于:

maximize performance

6 MDP —— Markov Decision Process

这一部分是后续大题核心，必须理解。

MDP = Markov Decision Process (马尔可夫决策过程)

用于:

- 在随机环境中做决策

- 状态转移带概率

MDP 定义为五元组:

$$(S, A, P, R, \gamma)$$

解释:

- S : 状态集合 (所有可能状态)
- A : 动作集合
- $P(s'|s, a)$: 从 s 执行动作 a 到达 s' 的概率
- $R(s, a)$: 奖励函数
- γ : 折扣因子 ($0 \leq \gamma < 1$)

为什么叫 Markov ?

因为满足:

$$P(s_{t+1}|s_t, s_{t-1}, \dots) = P(s_{t+1}|s_t)$$

未来只依赖当前状态。

7 Bellman Equation (贝尔曼方程)

价值函数:

$$V(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')]$$

解释:

当前价值 = 当前奖励 + 折扣后的未来最优价值

考试如何考 ?

给:

- 状态
- 转移概率
- 奖励
- γ

要求:

算一轮 value iteration。

步骤必须写:

1. 对每个 action 计算期望
2. 取最大

8 Q-learning (强化学习)

更新公式:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

解释:

- α : 学习率
- R : 即时奖励
- γ : 折扣

考试可能:

给数值, 让你算一次更新。

9 CSP (约束满足问题)

CSP = Constraint Satisfaction Problem

组成:

- Variables
- Domains
- Constraints

样题已经考:

- Forward Checking
- Constraint Propagation

Forward checking:

只检查已赋值变量的邻居。

Constraint propagation:

若某变量 domain 改变,

继续检查所有相关约束

10 Lecture 1 核心逻辑

整门课逻辑:

Perception $\rightarrow$ Model $\rightarrow$ Decision $\rightarrow$ Action

Search: 确定性决策

MDP: 随机决策

RL: 未知模型下学习决策

11 考试真正核心

已通过样题验证:

- UCS 计算路径
- Forward checking
- Constraint propagation

高概率未来计算题:

- Bellman 更新
- Value iteration
- Q-learning 单步更新

所有算法都在实现 Rational Decision-Making

Lecture 2: Linear Regression & Classification

12 Machine Learning Overview

基本模型：

$$x \rightarrow f \rightarrow y$$

- x : 输入 (features)
- f : 模型 (predictor)
- y : 输出 (label 或实数)

三类任务：

- Binary classification: $y \in \{+1, -1\}$
- Regression: $y \in \mathbb{R}$
- Structured prediction: y 为复杂结构

13 Linear Regression (线性回归)

1. Hypothesis Class

线性函数：

$$f_w(x) = w_1 + w_2x$$

向量形式：

$$\phi(x) = [1, x]$$

$$w = [w_1, w_2]$$

$$f_w(x) = w \cdot \phi(x)$$

Hypothesis class：

$$\mathcal{F} = \{f_w : w \in \mathbb{R}^2\}$$

解释：

- $\phi(x)$: feature vector
- w : 权重向量
- 点积表示线性组合

2. Loss Function

平方损失:

$$Loss(x, y, w) = (f_w(x) - y)^2$$

残差:

$$residual = f_w(x) - y$$

训练损失:

$$TrainLoss(w) = \frac{1}{|D_{train}|} \sum_{(x,y) \in D_{train}} (f_w(x) - y)^2$$

考试如何出计算题?

给数据点和 w , 要求计算 $TrainLoss(w)$ 。

步骤:

1. 计算每个 $f_w(x)$
2. 计算 residual
3. 平方
4. 求平均

3. Optimization

目标:

$$\min_w TrainLoss(w)$$

梯度:

$$\nabla_w TrainLoss(w) = \frac{1}{|D|} \sum 2(f_w(x) - y)\phi(x)$$

梯度下降更新:

$$w \leftarrow w - \eta \nabla_w \text{TrainLoss}(w)$$

其中 η 为 learning rate。

高概率计算题：

给初始 w 、数据、 η ，计算一次更新。

必须写完整向量计算过程。

14 Linear Classification (线性分类)

1. Hypothesis Class

预测函数：

$$f_w(x) = \text{sign}(w \cdot \phi(x))$$

决策边界：

$$w \cdot \phi(x) = 0$$

解释：

- 决策边界是一条直线
- 两侧分别预测 +1 与 -1

常见计算题：

给 w 与若干点，计算预测标签。

步骤：

1. 计算 $w \cdot \phi(x)$
2. 判断符号

2. Score 与 Margin

Score：

$$\text{score} = w \cdot \phi(x)$$

Margin：

$$\text{margin} = (w \cdot \phi(x))y$$

若 $\text{margin} > 0$ ，预测正确。

3. Zero-One Loss

$$Loss_{0-1}(x, y, w) = \mathbf{1}[(w \cdot \phi(x))y \leq 0]$$

解释:

预测错误损失为 1。

Zero-one loss 不可导, 不能直接做梯度下降。

4. Hinge Loss

$$Loss_{hinge}(x, y, w) = \max\{1 - (w \cdot \phi(x))y, 0\}$$

若 $margin \geq 1$, 损失为 0。

梯度:

$$\nabla_w = \begin{cases} -\phi(x)y & \text{if } 1 - (w \cdot \phi(x))y > 0 \\ 0 & \text{otherwise} \end{cases}$$

极高概率计算题:

给定点与 w :

- 计算 hinge loss
- 计算对应梯度

必须分情况写。

5. Logistic Loss

$$Loss_{logistic}(x, y, w) = \log(1 + e^{-(w \cdot \phi(x))y})$$

特点:

- 可导
- 平滑
- 梯度非零

15 Regression vs Classification 对比

	Regression	Classification
输出	$\mathbb{R}$	± 1
误差	residual	margin
损失	squared	hinge/logistic
优化	gradient descent	gradient descent

16 Lecture 2 真正考试核心

极高概率计算题：

- 计算 $TrainLoss$
- 计算 $\nabla_w TrainLoss$
- 做一次梯度更新
- 计算 hinge loss
- 计算分类预测

中概率：

- Logistic loss 形式
- score 与 margin 的关系

Lecture 2 = 计算梯度 + 参数更新

Lecture 3: Linear Classifier for Image Classification

17 Image Classification Problem

输入:

$$\text{Image} \in \mathbb{R}^{H \times W \times 3}$$

例如:

$$300 \times 500 \times 3$$

每个像素:

$$x_{i,j,c} \in [0, 255]$$

输出:

$$y \in \{1, 2, \dots, K\}$$

18 Image Classification Pipeline

三步:

1. 收集带标签数据
2. 训练分类器
3. 测试新图像

监督学习框架:

$$y = f(x)$$

训练集:

$$D = \{(x_1, y_1), \dots, (x_N, y_N)\}$$

19 Nearest Neighbor (NN)

预测规则：

$$f(x) = y_{i^*}$$

其中

$$i^* = \arg \min_i D(x, x_i)$$

Distance Metrics

L1 距离：

$$D_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

L2 距离：

$$D_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$

考试常问：

- 计算 L1 或 L2 距离
- 比较哪个更鲁棒

K-Nearest Neighbor

规则：

$$f(x) = \text{majority label among K nearest}$$

性质：

- 非参数模型
- 无训练阶段
- 测试时间复杂度 $O(N)$

调参：

- 选择 K

- 选择距离度量
- train / val / test 划分

20 Linear Classifier 回顾

二分类:

$$f(x) = \text{sign}(w \cdot \phi(x))$$

决策边界:

$$w \cdot \phi(x) = 0$$

加入 bias:

$$f(x) = w \cdot \phi(x) + b$$

其中:

$$b \in \mathbb{R}$$

21 Multi-class Linear Classifier

对于 K 类:

每类一个权重向量:

$$f_k(x) = w_k \cdot \phi(x) + b_k$$

堆叠为矩阵形式:

$$f(x) = W\phi(x) + b$$

维度:

$$W \in \mathbb{R}^{K \times d}$$

$$\phi(x) \in \mathbb{R}^d$$

预测:

$$\hat{y} = \arg \max_k f_k(x)$$

极高概率计算题：

- 给定 $W, b, \phi(x)$
- 计算所有 score
- 选最大类别

22 Training: Logistic Loss

单样本损失：

$$Loss(x, y, w) = \log(1 + e^{-y(w \cdot \phi(x))})$$

定义 sigmoid：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

梯度：

$$\nabla_w Loss = -\sigma(-yw \cdot \phi(x))\phi(x)y$$

考试会要求：

- 推导梯度
- 写出梯度下降更新

更新公式：

$$w \leftarrow w - \eta \nabla_w Loss$$

23 KNN vs Linear Classifier

	KNN	Linear Classifier
类型	Non-parametric	Parametric
训练	无	需要优化
测试时间	慢 $O(N)$	快 $O(dK)$
存储	存全部数据	只存参数

24 Non-linear Features

预测函数:

$$f_w(x) = w \cdot \phi(x)$$

线性于:

- w
- $\phi(x)$

非线性于:

- x

Quadratic Features

$$\phi(x) = [1, x, x^2]$$

预测集合:

$$\mathcal{F} = \{f_w(x) = w \cdot \phi(x) : w \in \mathbb{R}^3\}$$

Periodic Features

$$\phi(x) = [1, x, x^2, \cos(3x)]$$

维度:

$$w \in \mathbb{R}^4$$

关键思想:

- 通过改变 $\phi(x)$ 获得非线性决策边界
- 在 feature space 仍是线性超平面

25 Decision Boundary Example

若：

$$\phi(x) = [x_1, x_2, x_1^2 + x_2^2]$$

$$f(x) = \text{sign}([2, 2, -1] \cdot \phi(x))$$

则决策边界为圆：

$$(x_1 - 1)^2 + (x_2 - 1)^2 = 2$$

26 Lecture 3 核心总结

高概率：

- KNN 距离计算
- Multi-class score 计算
- Logistic loss 梯度
- 线性 vs 非线性区别

概念理解：

- Parametric vs Non-parametric
- Feature mapping 的作用
- 决策边界在 input space 与 feature space 的区别

Lecture 3 = 图像表示 + 多分类线性模型 + 特征非线性化

Lecture 4: Neural Networks and Backpropagation

27 From Linear to Non-linear Predictors

线性预测器：

$$f_w(x) = w \cdot \phi(x)$$

例如：

$$\phi(x) = (1, x)$$

二次特征：

$$\phi(x) = (1, x, x^2)$$

神经网络形式：

$$f(x) = w \cdot \sigma(V\phi(x))$$

其中：

- V ：隐藏层参数
- $\sigma(\cdot)$ ：非线性激活函数
- w ：输出层参数

28 Neural Network Structure

隐藏表示：

$$h(x) = \sigma(V\phi(x))$$

分类器：

$$f_{V,w}(x) = \text{sign}(w \cdot h(x))$$

理解：

- $h(x)$ 是学习到的特征表示
- 网络同时学习表示与分类器

29 Deep Neural Networks

1 层:

$$\text{score} = w \cdot \phi(x)$$

2 层:

$$\text{score} = w \cdot \sigma(V\phi(x))$$

3 层:

$$\text{score} = w \cdot \sigma(V_2\sigma(V_1\phi(x)))$$

一般形式:

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)})$$

30 Why Depth?

直觉:

- 多层抽象表示
- 建模复杂关系
- 经验效果好

31 Training a 2-Layer Network

前向传播:

$$h = \sigma(XW_1)$$

$$\hat{y} = hW_2$$

平方损失:

$$L = \|\hat{y} - y\|^2$$

梯度:

$$\begin{aligned}\nabla_{W_2} &= h^T(\hat{y} - y) \\ \nabla_{W_1} &= X^T ((\hat{y} - y)W_2^T \odot h(1 - h))\end{aligned}$$

参数更新:

$$W \leftarrow W - \eta \nabla_W$$

考试重点:

- 写出前向传播
- 写出梯度表达式
- 给出更新公式

32 Why Backpropagation?

复杂网络直接手推:

- 极其繁琐
- 换 loss 需要重新推导

解决方法:

Computational Graph + Chain Rule

33 Backpropagation Core Idea

设:

$$z = f(x, y)$$

上游梯度:

$$\frac{\partial L}{\partial z}$$

链式法则:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}$$

$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial y}$$

规则:

- 上游梯度 $\times$ 局部梯度

34 Simple Example

$$f(x, y, z) = (x + y)z$$

设:

$$q = x + y$$

局部梯度:

$$\frac{\partial q}{\partial x} = 1$$

$$\frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial q} = z$$

$$\frac{\partial f}{\partial z} = q$$

链式法则:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = z$$

$$\frac{\partial f}{\partial y} = z$$

35 Gradient Flow Patterns

Add Gate

$$q = x + y$$

梯度分配：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial q}$$

$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial q}$$

Multiply Gate

$$q = xy$$

梯度交换：

$$\begin{aligned}\frac{\partial L}{\partial x} &= \frac{\partial L}{\partial q} \cdot y \\ \frac{\partial L}{\partial y} &= \frac{\partial L}{\partial q} \cdot x\end{aligned}$$

Max Gate

$$q = \max(x, y)$$

梯度路由到较大者。

36 Backprop Example with Sigmoid

$$L = \sigma(w_0x_0 + w_1x_1 + w_2)$$

Sigmoid：

$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

导数：

$$\sigma'(s) = \sigma(s)(1 - \sigma(s))$$

设：

$$s = w_0x_0 + w_1x_1 + w_2$$

梯度：

$$\begin{aligned}\frac{\partial L}{\partial s} &= \sigma(s)(1 - \sigma(s)) \\ \frac{\partial L}{\partial w_0} &= \frac{\partial L}{\partial s} x_0 \\ \frac{\partial L}{\partial x_0} &= \frac{\partial L}{\partial s} w_0\end{aligned}$$

高概率考点：

- 手算 sigmoid 梯度
- 乘法节点梯度传播
- 写出完整反向传播步骤

37 Lecture 4 总结

必须掌握：

- 多层网络表达式
- 链式法则
- add / multiply 梯度模式
- sigmoid 导数
- 参数更新公式

核心思想：

Backprop = 上游梯度 × 局部梯度

Neural Networks = 学习分层特征表示

Lecture 5: Unsupervised Learning

38 Supervised vs Unsupervised Learning

监督学习:

训练数据

$$D_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n$$

目标:

$$f_w(x) = w \cdot \phi(x)$$

使用真实标签 y 计算 loss。

问题: 标注数据昂贵。

无监督学习:

训练数据

$$D_{\text{train}} = \{x_i\}_{i=1}^n$$

没有标签 y 。

无标签数据非常便宜。

39 Clustering Task

定义: 聚类

输入:

$$D_{\text{train}} = \{x_1, \dots, x_n\}$$

输出:

$$z = (z_1, \dots, z_n), \quad z_i \in \{1, \dots, K\}$$

目标:

希望相近的点属于同一类。

40 Centroids

每个簇由一个中心表示:

$$\mu_k \in \mathbb{R}^d, \quad k = 1, \dots, K$$

所有中心:

$$\mu = (\mu_1, \dots, \mu_K)$$

目标:

x_i 应接近 μ_{z_i}

41 K-means Objective

目标函数:

$$\text{Loss}_{\text{kmeans}}(z, \mu) = \sum_{i=1}^n \|x_i - \mu_{z_i}\|^2$$

优化问题:

$$\min_z \min_{\mu} \text{Loss}_{\text{kmeans}}(z, \mu)$$

两件事:

- 找最佳中心 μ
- 给每个点分配最近中心

42 Alternating Minimization

K-means 使用交替最小化。

Step 1: Fix μ , update z

$$z_i = \arg \min_{k=1, \dots, K} \|x_i - \mu_k\|^2$$

即: 分配到最近中心。

Step 2: Fix z , update μ

$$\mu_k = \frac{1}{|\{i : z_i = k\}|} \sum_{i: z_i = k} x_i$$

即: 取均值。

考试重点:

- 写出 K-means loss

- 写出 assignment 更新公式
- 写出 centroid 更新公式

43 K-means Algorithm

初始化:

$\mu_1, \dots, \mu_K$ 随机初始化

重复直到收敛:

- 更新 z_i
- 更新 μ_k

收敛条件:

z 不再变化

44 Gradient Descent Review

训练损失:

$$\text{TrainLoss}(w) = \frac{1}{|D_{\text{train}}|} \sum_{(x,y) \in D_{\text{train}}} \text{Loss}(x, y, w)$$

梯度下降:

$$w \leftarrow w - \eta \nabla_w \text{TrainLoss}(w)$$

问题:

每次迭代都要遍历全部数据, 代价大。

45 Stochastic Gradient Descent

SGD 更新:

$$w \leftarrow w - \eta \nabla_w \text{Loss}(x_i, y_i, w)$$

特点:

- 每次只用一个样本

- 更快
- 梯度更噪声

46 Mini-batch SGD

使用小批量 B :

$$w \leftarrow w - \eta \frac{1}{|B|} \sum_{(x,y) \in B} \nabla_w \text{Loss}(x, y, w)$$

折中: 更稳定 + 仍然高效。

47 Generalization

关键问题:

训练 loss 是否等于真实目标?

真实目标:

Minimize error on unseen data

定义: 测试集

D_{test} 不参与训练

48 Overfitting

现象:

- 训练误差下降
- 验证误差上升

过拟合 = 模型太复杂

49 Strategy 1: Regularization

正则化目标：

$$\min_w \text{TrainLoss}(w) + \frac{\lambda}{2} \|w\|^2$$

梯度：

$$\nabla_w = \nabla_w \text{DataLoss} + \lambda w$$

作用：

限制模型复杂度。

50 Strategy 2: Early Stopping

思想：

减少训练轮数 T

经验：

不要把训练误差压到极致。

51 Hyperparameters

定义：

在训练前决定的设计选择。

包括：

- 特征 $\phi(x)$
- loss 类型 (hinge, logistic)
- 正则化强度 λ
- 学习率 η
- batch size
- 训练轮数

52 Validation Set

数据划分：

$$D_{\text{train}}, \quad D_{\text{val}}, \quad D_{\text{test}}$$

步骤：

- 在 D_{train} 训练
- 在 D_{val} 选超参数
- 最终只在 D_{test} 报告结果

测试集绝对不能用于调参。

53 K-Fold Cross Validation

步骤：

- 将数据分成 K 份
- 每次用 $K - 1$ 份训练
- 1 份验证
- 计算平均误差

作用：

提高评估稳定性

54 Lecture 5 Summary

必须掌握：

- K-means 目标函数
- Alternating minimization
- SGD 与 mini-batch
- 过拟合与泛化

- 正则化公式
- Validation / Test 区别

核心思想:

真实目标 = 泛化误差

Keep the model simple

Lecture 5: Unsupervised Learning

55 Supervised vs Unsupervised Learning

监督学习：

训练数据

$$D_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n$$

目标：

$$f_w(x) = w \cdot \phi(x)$$

使用真实标签 y 计算 loss。

问题：标注数据昂贵。

无监督学习：

训练数据

$$D_{\text{train}} = \{x_i\}_{i=1}^n$$

没有标签 y 。

无标签数据非常便宜。

56 Clustering Task

定义：聚类

输入：

$$D_{\text{train}} = \{x_1, \dots, x_n\}$$

输出：

$$z = (z_1, \dots, z_n), \quad z_i \in \{1, \dots, K\}$$

目标：

希望相近的点属于同一类。

57 Centroids

每个簇由一个中心表示：

$$\mu_k \in \mathbb{R}^d, \quad k = 1, \dots, K$$

所有中心：

$$\mu = (\mu_1, \dots, \mu_K)$$

目标:

x_i 应接近 μ_{z_i}

58 K-means Objective

目标函数:

$$\text{Loss}_{\text{kmeans}}(z, \mu) = \sum_{i=1}^n \|x_i - \mu_{z_i}\|^2$$

优化问题:

$$\min_z \min_{\mu} \text{Loss}_{\text{kmeans}}(z, \mu)$$

两件事:

- 找最佳中心 μ
- 给每个点分配最近中心

59 Alternating Minimization

K-means 使用交替最小化。

Step 1: Fix μ , update z

$$z_i = \arg \min_{k=1, \dots, K} \|x_i - \mu_k\|^2$$

即: 分配到最近中心。

Step 2: Fix z , update μ

$$\mu_k = \frac{1}{|\{i : z_i = k\}|} \sum_{i: z_i = k} x_i$$

即: 取均值。

考试重点:

- 写出 K-means loss

- 写出 assignment 更新公式
- 写出 centroid 更新公式

60 K-means Algorithm

初始化:

$\mu_1, \dots, \mu_K$ 随机初始化

重复直到收敛:

- 更新 z_i
- 更新 μ_k

收敛条件:

z 不再变化

61 Gradient Descent Review

训练损失:

$$\text{TrainLoss}(w) = \frac{1}{|D_{\text{train}}|} \sum_{(x,y) \in D_{\text{train}}} \text{Loss}(x, y, w)$$

梯度下降:

$$w \leftarrow w - \eta \nabla_w \text{TrainLoss}(w)$$

问题:

每次迭代都要遍历全部数据，代价大。

62 Stochastic Gradient Descent

SGD 更新:

$$w \leftarrow w - \eta \nabla_w \text{Loss}(x_i, y_i, w)$$

特点:

- 每次只用一个样本

- 更快
- 梯度更噪声

63 Mini-batch SGD

使用小批量 B :

$$w \leftarrow w - \eta \frac{1}{|B|} \sum_{(x,y) \in B} \nabla_w \text{Loss}(x, y, w)$$

折中: 更稳定 + 仍然高效。

64 Generalization

关键问题:

训练 loss 是否等于真实目标?

真实目标:

Minimize error on unseen data

定义: 测试集

D_{test} 不参与训练

65 Overfitting

现象:

- 训练误差下降
- 验证误差上升

过拟合 = 模型太复杂

66 Strategy 1: Regularization

正则化目标：

$$\min_w \text{TrainLoss}(w) + \frac{\lambda}{2} \|w\|^2$$

梯度：

$$\nabla_w = \nabla_w \text{DataLoss} + \lambda w$$

作用：

限制模型复杂度。

67 Strategy 2: Early Stopping

思想：

减少训练轮数 T

经验：

不要把训练误差压到极致。

68 Hyperparameters

定义：

在训练前决定的设计选择。

包括：

- 特征 $\phi(x)$
- loss 类型 (hinge, logistic)
- 正则化强度 λ
- 学习率 η
- batch size
- 训练轮数

69 Validation Set

数据划分：

$$D_{\text{train}}, \quad D_{\text{val}}, \quad D_{\text{test}}$$

步骤：

- 在 D_{train} 训练
- 在 D_{val} 选超参数
- 最终只在 D_{test} 报告结果

测试集绝对不能用于调参。

70 K-Fold Cross Validation

步骤：

- 将数据分成 K 份
- 每次用 $K - 1$ 份训练
- 1 份验证
- 计算平均误差

作用：

提高评估稳定性

71 Lecture 5 Summary

必须掌握：

- K-means 目标函数
- Alternating minimization
- SGD 与 mini-batch
- 过拟合与泛化

- 正则化公式
- Validation / Test 区别

核心思想:

真实目标 = 泛化误差

Keep the model simple

Lecture 6: Uninformed Search

72 Search Problem Definition

一个搜索问题由以下部分定义：

- State space (状态空间)
- Successor function (后继函数)
- Start state (初始状态)
- Goal test (目标判定)

后继函数：

输入：

(state, action)

输出：

(next state, cost)

Solution:

一系列 action, 使得 start state $\rightarrow$ goal state

73 General Tree Search

核心组件：

- Frontier: 当前待扩展节点集合
- Expanded: 已经扩展的节点
- Unexplored: 未访问区域

关系：

$$\text{Reached} = \text{Expanded} \cup \text{Frontier}$$

关键问题：

Which node to expand next?

不同策略 = 不同搜索算法。

74 Uninformed Search

定义：

只使用问题本身提供的信息，不使用额外启发信息。

包括：

- Depth-First Search (DFS)
- Breadth-First Search (BFS)
- Iterative Deepening Search (IDS)
- Uniform-Cost Search (UCS)

75 Depth-First Search (DFS)

策略：

Expand the deepest node first

可用数据结构：

Stack (LIFO)

复杂度分析

设：

- b : branching factor
- m : 最大深度

时间复杂度：

$$O(b^m)$$

空间复杂度：

$$O(bm)$$

性质

- Complete?

No, 若 $m = \infty$

- Optimal?

No, 可能找到较深或高代价解

76 Breadth-First Search (BFS)

策略:

Expand the shallowest node first

数据结构:

Queue (FIFO)

复杂度分析

设:

- s : 最浅解的深度

时间复杂度:

$O(b^s)$

空间复杂度:

$O(b^s)$

性质

Complete?

Yes, 若 $s < \infty$

Optimal?

Yes, 当每步代价相同时

77 DFS vs BFS

考试常问:

什么时候 BFS 更好?

当解在较浅层时

什么时候 DFS 更好?

当内存受限或解较深时

对比:

DFS 空间 $O(bm)$, BFS 空间 $O(b^s)$

78 Iterative Deepening Search (IDS)

思想:

结合 DFS 的低空间 + BFS 的浅层优势

算法:

- 先做 depth limit = 1
- 若失败, limit = 2
- 若失败, limit = 3
- 直到找到解

复杂度

时间复杂度:

$$O(b^s)$$

空间复杂度:

$$O(bs)$$

性质

- Complete: Yes
- Optimal: Yes (单位代价)

79 Uniform-Cost Search (UCS)

定义:

$g(n)$ = 从起点到 n 的路径代价

策略:

Expand node with smallest $g(n)$

数据结构:

Priority Queue

复杂度

设:

- C^* : 最优解代价
- ϵ : 最小步代价

等效深度:

$$\frac{C^*}{\epsilon}$$

时间复杂度:

$$O(b^{C^*/\epsilon})$$

空间复杂度:

$$O(b^{C^*/\epsilon})$$

性质

Complete?

Yes, 若 $C^* < \infty$ 且 $\epsilon > 0$

Optimal?

Yes

80 Lecture 6 总结

必须掌握:

- Search problem 四要素
- Frontier / Expanded 概念
- DFS 时间与空间复杂度
- BFS 时间与空间复杂度
- IDS 思想
- UCS 复杂度推导
- Complete vs Optimal 区别

核心对比:

DFS: 深度优先, 省空间, 不保证最优

BFS: 层级优先, 保证最浅解, 耗空间

UCS: 代价优先, 保证最优解

Lecture 7: Informed Search

81 Informed vs. Uninformed Search

Recall:

- DFS: deepest node first
- BFS: shallowest node first
- UCS: lowest path cost $g(n)$

Uninformed search:

Only uses information from problem definition

Informed search:

Use problem-specific heuristic knowledge

Guide search toward the goal

82 Greedy Search

核心思想:

Expand node that seems closest to goal

需要定义启发函数:

$h(n)$ = estimated cost from n to goal

选择规则:

Pick node with smallest $h(n)$

Example: Route Finding

Romania 地图问题:

$$h(n) = \text{straight-line distance to Bucharest}$$

特点:

- Problem-specific
- Useful but may be inaccurate

Greedy 性质

Complete?

No (可能陷入 cycle)

Optimal?

No (只看 $h(n)$, 忽略 $g(n)$)

83 From Greedy to A*

Greedy 只考虑:

$$h(n)$$

UCS 只考虑:

$$g(n)$$

自然想法:

Combine them

84 A* Search

定义:

$$f(n) = g(n) + h(n)$$

其中:

- $g(n)$: 从起点到 n 的真实代价
- $h(n)$: 从 n 到目标的估计代价

选择规则:

Expand node with smallest $f(n)$

极端情况

$g(n) \rightarrow 0 \Rightarrow$ Greedy Search

$h(n) \rightarrow 0 \Rightarrow$ Uniform-Cost Search

85 Is A* Optimal?

若 h 可能高估:

A* may choose suboptimal solution

问题核心:

Actual cost < Estimated cost

因此我们需要:

Heuristic must not overestimate

86 Admissible Heuristic

定义:

$$h(n) \leq h^*(n)$$

其中:

$h^*(n)$ = true cost from n to goal

等价表达:

Never overestimate

Examples

- Straight-line distance
- Manhattan distance

87 Optimality of A* (Tree Search)

假设:

- A 是 optimal goal
- B 是 suboptimal goal
- h admissible

目标:

A is expanded before B

Proof Sketch

设 n 是 A 的某个 ancestor。

第一步:

$$f(n) \leq f(A)$$

因为:

$$f(n) = g(n) + h(n)$$

$$h(n) \leq h^*(n)$$

$$g(n) + h^*(n) = g(A)$$

而目标节点:

$$f(A) = g(A)$$

第二步:

$$f(A) < f(B)$$

因为:

$$g(A) < g(B)$$

且:

$$h(A) = h(B) = 0$$

因此:

$$f(n) \leq f(A) < f(B)$$

结论:

A^* tree search is optimal

88 Heuristic Design

重要思想:

Admissible heuristics often come from relaxed problems

如果放宽约束:

Cost of relaxed solution $\leq$ true cost

因此是 admissible。

89 8-Puzzle Heuristics

Misplaced Tiles

$h_1(n)$ = number of misplaced tiles

Admissible, 因为:

Each misplaced tile needs at least one move

Manhattan Distance

$$h_2(n) = \sum \text{Manhattan distance of each tile}$$

更精确，因此搜索更快。

90 Heuristic Dominance

定义：

$$h_1 \geq h_2$$

若：

$$\forall n, h_1(n) \geq h_2(n)$$

且两者都 admissible。

性质：

Larger admissible heuristic is better

特殊情况：

$$h(n) = 0 \Rightarrow A^* \text{ becomes UCS}$$

91 Combining Heuristics

若有多个 admissible heuristics：

$$h_1(n), \dots, h_m(n)$$

构造：

$$h(n) = \max\{h_1(n), \dots, h_m(n)\}$$

性质：

Max of admissible heuristics is admissible

且：

$$h \text{ dominates each } h_i$$

92 Properties of A*

Optimal?

Yes, if h is admissible

Complete?

Yes

Time Complexity

Exponential in solution length

Space Complexity

Exponential

93 Lecture 7 总结

考试必考点:

- Greedy 定义与缺点
- A* 定义
- $f(n) = g(n) + h(n)$
- Admissible 定义
- A* 最优性证明思路
- Heuristic dominance
- Max 组合 heuristic

核心公式:

$$f(n) = g(n) + h(n)$$

$$h(n) \leq h^*(n)$$

max of admissible heuristics is admissible

Lecture 8: Constraint Satisfaction Problems

94 What is Search For?

Planning problems (规划问题)

- Sequence of actions (动作序列)
- Path to goal is important (路径重要)
- Paths have costs and depths (有代价和深度)
- Heuristics guide search (启发函数指导搜索)

Identification problems (识别型问题)

- Assignments to variables (变量赋值)
- Goal itself is important (目标状态重要)
- Path is irrelevant (路径不重要)

CSPs are specialized for identification problems

95 CSP Definition (形式定义)

A CSP is a special subset of search problems

State (状态)

Defined by variables $X_1, X_2, \dots, X_n$

Each variable has domain:

D_i = set of possible values

Goal (目标)

Find a complete assignment satisfying all constraints

Goal test (目标检测)

Check whether constraints are all satisfied

96 Example: Map Coloring (地图染色)

Variables (变量) :

$$WA, NT, Q, NSW, V, SA, T$$

Domains (取值域) :

$$D = \{red, green, blue\}$$

Constraint (约束) :

Adjacent regions must have different colors

Example solution (示例解) :

$$\{WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green\}$$

关键点:

We care about the assignment, not the path

97 Types of Constraints (约束类型)

Unary constraint (一元约束)

$$SA \neq green$$

Binary constraint (二元约束)

$$SA \neq WA$$

Binary constraints form a graph:

Nodes = variables

Edges = constraints

Higher-order constraint (高阶约束)

Involve 3 or more variables

98 Standard Search Formulation

State space (状态空间) :

Partial assignments

Initial state (初始状态) :

$\{\}$

Successor (扩展方式) :

Assign value to one unassigned variable

Goal test:

Assignment complete and consistent

99 Naive DFS Complexity

Let:

n = number of variables

d = domain size

Naive branching:

$$b = nd$$

Search tree size:

$$O((nd)^n)$$

Huge and impractical

100 Backtracking Search (回溯搜索)

核心思想:

Assign one variable at a time

Branching factor:

$$b = d$$

Tree size:

$$O(d^n)$$

改进点:

Only try values consistent with previous assignments

Backtrack when no legal values remain

Backtracking = DFS + consistency checking

101 Ordering Heuristics (排序启发式)

Two questions:

Which variable next?

Which value first?

101.1 Minimum Remaining Values (MRV)

定义:

Choose variable with fewest legal values

中文理解:

优先处理最容易失败的变量

Also called:

Most constrained variable

目的:

Fail early, prune early

101.2 Least Constraining Value (LCV)

定义:

Choose value that eliminates fewest choices of neighbors

中文理解:

给别人留更多选择

目的:

Preserve future flexibility

102 Filtering (过滤)

核心思想:

Detect inevitable failure as early as possible

102.1 Forward Checking

当变量被赋值时:

Remove inconsistent values from neighbors

如果某个域为空:

Immediate failure

局限:

Only propagates from assigned variables

102.2 Arc Consistency (弧一致性)

定义:

Arc $X \rightarrow Y$ is consistent if

$$\forall x \in D_X, \exists y \in D_Y$$

such that constraint holds.

如果不存在:

Remove x from D_X

重要性质:

If D_X changes, neighbors must be rechecked

Arc consistency is stronger than forward checking

103 Iterative Improvement (局部搜索法)

不同思路:

Work with complete assignments

允许:

Some constraints unsatisfied

通过修改变量值减少冲突。

103.1 Min-Conflicts

Step 1:

Randomly pick a conflicted variable

Step 2:

Assign value minimizing conflicts

特点:

Very effective for large CSPs

104 Lecture 8 Summary

CSP 核心特征:

- States are partial assignments

- Goal defined by constraints
- Path is irrelevant

三类优化:

- Ordering: MRV + LCV
- Filtering: Forward checking
- Propagation: Arc consistency

考试高频对比:

MRV vs LCV

Forward checking vs Arc consistency

Backtracking vs Min-conflicts

Lecture 9: Local Search Algorithms

105 Introduction to Local Search

Motivation (动机)

In many optimization problems, path is irrelevant

The goal state itself is the solution

典型例子:

n-queens: find a configuration with no conflicts

TSP: find shortest tour visiting each city once

状态空间:

State space = set of complete configurations

核心思想:

Keep a single current state and iteratively improve it

Local search works on complete assignments

106 Local Search Characteristics

局部搜索算法:

Operate on a single current node

Move only to neighboring states

优点:

Very low memory usage (often constant)

Effective in very large state spaces

107 Hill Climbing Algorithm (爬山算法)

基本思想:

Start with an arbitrary state

Repeatedly move to the best neighbor

Stop if no neighbor is better

形式化描述:

At each step choose $s' = \arg \max_{neighbor} f(s')$

其中:

$f(s)$ = objective function

Hill climbing only looks at immediate neighbors

107.1 State Space Landscape (状态空间地形)

可以将问题理解为:

Maximize objective function over state space

重要概念:

Global maximum: best overall solution

Local maximum: better than neighbors but not global

Plateau: flat region of equal value

问题:

Hill climbing may get stuck in local maxima

Initial state strongly affects result

107.2 Hill Climbing on n-Queens

定义启发函数:

$$h(s) = \text{number of attacking pairs}$$

目标:

$$\min h(s)$$

现象:

May reach a local minimum where all neighbors are worse

Hill climbing is not complete and not optimal

107.3 Escaping Local Optima

最简单方法:

Random restart

思想:

Run hill climbing multiple times from random states

108 Simulated Annealing (模拟退火)

物理类比:

Annealing: heat and slowly cool material

算法思想:

Allow bad moves with some probability

关键变量:

$$T = \text{temperature}$$

算法步骤:

$$\Delta E = f(next) - f(current)$$

If $\Delta E > 0$ accept move

否则以概率接受:

$$P = e^{\Delta E/T}$$

性质:

$$0 \leq e^{\Delta E/T} \leq 1$$

重要理解:

Large $T \Rightarrow$ bad moves likely

$T \rightarrow 0 \Rightarrow$ behaves like hill climbing

If temperature decreases slowly enough, can approximate global optimum

109 Local Beam Search (局部束搜索)

基本思想:

Maintain K states instead of one

每轮:

Generate all successors of K states

Select best K among them

关键区别:

Search processes communicate

直觉类比:

Similar to evolution

110 Genetic Algorithm (遗传算法)

表示方式:

State represented as a string

例如 8-queens:

[1, 6, 2, 5, 7, 4, 8, 3]

初始化:

Generate K random states (population)

110.1 Fitness Function (适应度函数)

与启发函数区别:

Fitness: higher is better

8-queens:

Fitness = number of non-attacking pairs

最大值:

$$\frac{8 \times 7}{2} = 28$$

选择概率:

$$Pr(i) = \frac{Fitness(i)}{\sum_j Fitness(j)}$$

Also called roulette-wheel selection

110.2 Genetic Operators

Crossover (交叉)

Combine two parents to produce offspring

类型:

Single-point crossover

Two-point crossover

Mutation (变异)

Randomly modify string with small probability

作用:

Maintain diversity

111 Genetic Algorithm Procedure

Step 1: Encode states

Step 2: Initialize population

Step 3: Evaluate fitness

Step 4: Selection

Step 5: Crossover

Step 6: Mutation

Repeat until termination

112 Lecture 9 Summary

Local search is suitable when path is irrelevant

四类算法:

- Hill climbing
- Simulated annealing

- Local beam search
- Genetic algorithm

关键对比:

Hill climbing vs Simulated annealing

Single state vs Multiple states

Fitness vs Heuristic cost

Important concept:

Many ML algorithms (e.g., gradient descent) are local search

Lecture 10: Markov Decision Process I

113 From Search to MDP

So far: Deterministic Search

State s , Action $a \rightarrow$ Successor s'

$$Succ(s, a)$$

特点:

Taking action a always leads to a fixed s'

Uncertainty in real world

State s , Action $a \rightarrow$ multiple possible s'

Transition is stochastic (random)

Need a framework to handle non-deterministic environments

114 Definition of Markov Decision Process

Definition: MDP

一个 MDP 定义为六元组:

$$(\text{States}, s_{start}, \text{Actions}, T, \text{Reward}, IsEnd)$$

组成部分:

- States: 状态集合
- $s_{start} \in \text{States}$: 初始状态
- $\text{Actions}(s)$: 在状态 s 可采取的动作
- $T(s, a, s')$: 转移概率
- $\text{Reward}(s, a, s')$: 即时奖励
- $IsEnd(s)$: 是否终止状态

For each state-action pair:

$$\sum_{s'} T(s, a, s') = 1$$

115 MDP vs Search Problem

主要区别:

$$Succ(s, a) \rightarrow T(s, a, s')$$

$$Cost(s, a) \rightarrow Reward(s, a, s')$$

Search returns a path

MDP returns a policy

116 Utility and Discounting

一条路径:

$$s_0, a_1, r_1, s_1, a_2, r_2, s_2, \dots$$

Discounted utility:

$$U = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

其中:

$$0 \leq \gamma \leq 1$$

解释:

- $\gamma = 1$: 不折扣未来
- $\gamma = 0$: 只关心当前奖励
- $0 < \gamma < 1$: 指数衰减

Discount ensures convergence when horizon is infinite

117 Policy

Definition: Policy

$$\pi : States \rightarrow Actions$$

$$\pi(s) \in Actions(s)$$

解释:

Policy tells us what action to take at each state

118 Policy Evaluation

Value of a policy:

$$V_{\pi}(s) = \text{Expected utility starting from } s$$

Q-value of a policy:

$$Q_{\pi}(s, a) = \text{Expected utility of taking } a \text{ then following } \pi$$

关系:

$$V_{\pi}(s) = \begin{cases} 0 & \text{if } IsEnd(s) \\ Q_{\pi}(s, \pi(s)) & \text{otherwise} \end{cases}$$

118.1 Bellman Expectation Equation

$$Q_{\pi}(s, a) = \sum_{s'} T(s, a, s') (Reward(s, a, s') + \gamma V_{\pi}(s'))$$

因此:

$$V_{\pi}(s) = \sum_{s'} T(s, \pi(s), s') (Reward(s, \pi(s), s') + \gamma V_{\pi}(s'))$$

This is a system of linear equations

119 Iterative Policy Evaluation

当状态很多时不能直接解线性方程组。

Key idea: Iterative update

初始化：

$$V_{\pi}^{(0)}(s) = 0$$

迭代：

$$V_{\pi}^{(t)}(s) = \sum_{s'} T(s, \pi(s), s') (Reward(s, \pi(s), s') + \gamma V_{\pi}^{(t-1)}(s'))$$

停止条件：

$$\max_s |V_{\pi}^{(t)}(s) - V_{\pi}^{(t-1)}(s)| \leq \epsilon$$

复杂度：

$$O(t_{PE} \cdot S \cdot S')$$

其中：

- S : 状态数
- S' : 每个状态的后继数

120 Optimal Value

目标：

$$V_{opt}(s) = \max_{\pi} V_{\pi}(s)$$

递归定义：

$$V_{opt}(s) = \begin{cases} 0 & \text{if } IsEnd(s) \\ \max_{a \in Actions(s)} Q_{opt}(s, a) & \end{cases}$$

其中：

$$Q_{opt}(s, a) = \sum_{s'} T(s, a, s') (Reward(s, a, s') + \gamma V_{opt}(s'))$$

This is the Bellman Optimality Equation

121 Value Iteration Algorithm

初始化:

$$V_{opt}^{(0)}(s) = 0$$

迭代更新:

$$V_{opt}^{(t)}(s) = \max_{a \in \text{Actions}(s)} \sum_{s'} T(s, a, s') \left(\text{Reward}(s, a, s') + \gamma V_{opt}^{(t-1)}(s') \right)$$

停止条件:

$$\max_s \left| V_{opt}^{(t)}(s) - V_{opt}^{(t-1)}(s) \right| \leq \epsilon$$

复杂度:

$$O(t_{VI} \cdot S \cdot A \cdot S')$$

Need to loop over all actions for each state

122 Extracting Optimal Policy

当 V_{opt} 已收敛后:

$$\pi_{opt}(s) = \arg \max_{a \in \text{Actions}(s)} Q_{opt}(s, a)$$

即:

$$\pi_{opt}(s) = \arg \max_{a \in \text{Actions}(s)} \sum_{s'} T(s, a, s') \left(\text{Reward}(s, a, s') + \gamma V_{opt}(s') \right)$$

123 Lecture 10 Summary

MDP handles uncertainty with probabilities

Solution of MDP = policy

两类核心算法:

- Policy Evaluation: $(MDP, \pi) \rightarrow V_\pi$
- Value Iteration: $MDP \rightarrow (V_{opt}, \pi_{opt})$

核心三条公式必须掌握：

$$Q_{\pi}(s, a)$$

$$V_{\pi}(s)$$

$$V_{opt}(s)$$

Lecture 11: Markov Decision Process II

124 Roadmap

- Value Iteration
- Policy Iteration
- A Preview of Reinforcement Learning

125 Recap of MDP

MDP 的四个核心量:

- Policy: $\pi(s)$
- Utility: 折扣奖励和
- Value: $V(s)$
- Q-Value: $Q(s, a)$

Search returns a path, MDP returns a policy

126 Value Iteration Review

Bellman Optimality Equation:

$$V_{opt}(s) = \max_{a \in Actions(s)} \sum_{s'} T(s, a, s') (Reward(s, a, s') + \gamma V_{opt}(s'))$$

Value Iteration 更新:

$$V^{(t)}(s) = \max_a \sum_{s'} T(s, a, s') (Reward(s, a, s') + \gamma V^{(t-1)}(s'))$$

复杂度:

$$O(t_{VI} S A S')$$

127 Problems with Value Iteration

- Slow: 每次都要对所有 action 取 max
- “max” rarely changes
- Policy often converges earlier than values

Could we directly solve for the policy?

答案:

Yes — Policy Iteration

128 Policy Iteration

Key idea:

- Policy Evaluation
- Policy Improvement
- Repeat until convergence

128.1 Policy Evaluation

给定固定策略 π , 求:

$$V_{\pi}(s)$$

Bellman Expectation Equation:

$$V_{\pi}(s) = \sum_{s'} T(s, \pi(s), s') (Reward(s, \pi(s), s') + \gamma V_{\pi}(s'))$$

两种计算方法:

- 解线性方程组
- Iterative policy evaluation (Lecture 10)

若用迭代法:

$$V_{\pi}^{(t)}(s) = \sum_{s'} T(s, \pi(s), s') (Reward(s, \pi(s), s') + \gamma V_{\pi}^{(t-1)}(s'))$$

复杂度:

$$O(t_{PE}SS')$$

注意:

No max over actions here

128.2 Policy Improvement

给定 V_π , 改进策略:

$$\pi_{new}(s) = \arg \max_a \sum_{s'} T(s, a, s') (Reward(s, a, s') + \gamma V_\pi(s'))$$

称为:

One-step lookahead

复杂度:

$$O(SAS')$$

128.3 Policy Iteration Algorithm

初始化:

$$\pi_0(s) \text{ arbitrary}$$

重复:

- Step 1: Policy Evaluation $\rightarrow$ compute V_{π_i}
- Step 2: Policy Improvement $\rightarrow$ get π_{i+1}

停止条件:

$$\pi_{i+1} = \pi_i$$

返回:

$$\pi^* = \pi_i$$

总复杂度:

$$O(t_{PI}(t_{PE}SS' + SAS'))$$

In practice, few iterations needed

129 Value Iteration vs Policy Iteration

Value Iteration:

- 每轮同时更新 value 和 policy
- 不显式存 policy

Policy Iteration:

- 多次 value update (固定 policy)
- 再做一次 policy update
- 通常收敛更快

核心区别:

VI: improve values directly

PI: improve policy directly

130 MDP Algorithms Summary

Compute optimal values:

- Value Iteration
- Policy Iteration

Compute value of fixed policy:

$$(MDP, \pi) \rightarrow V_\pi$$

Extract policy from value:

$$V \rightarrow \pi$$

131 Offline vs Online Planning

131.1 Offline Planning

已知：

- $T(s, a, s')$
- $Reward(s, a, s')$

通过计算得到：

$$\pi^*$$

Do not need to interact with environment

131.2 Online Planning

若未知转移概率：

$$T(s, a, s') \text{ unknown}$$

必须：

$$\text{Act} \rightarrow \text{Observe} \rightarrow \text{Learn}$$

这就是：

Reinforcement Learning

132 Preview: Reinforcement Learning

区别：

- MDP: model known
- RL: model unknown

目标仍然是：

$$\max_{\pi} V_{\pi}(s)$$

但方法不同：

Learn from interaction

133 Lecture 11 Summary

- Policy Iteration = Policy Evaluation + Policy Improvement
- Often converges faster than Value Iteration
- Offline planning assumes known model
- Reinforcement Learning handles unknown model

Next: Reinforcement Learning

Lecture 12: Reinforcement Learning I

134 Introduction to Reinforcement Learning

134.1 From MDP to RL

在 MDP 中我们假设：

- $T(s, a, s')$ 已知
- $Reward(s, a, s')$ 已知

但现实中：

$$T(s, a, s') \text{ unknown}$$

$$Reward(s, a, s') \text{ unknown}$$

这就是：

Reinforcement Learning

134.2 Basic Idea of RL

Reinforcement Learning 核心思想：

- 与环境交互 (interact with environment)
- 获得奖励反馈 (receive reward)
- 学习最大化期望累计奖励

目标仍然是：

$$\max_{\pi} V_{\pi}(s)$$

区别在于：

All learning is based on observed samples

135 Offline vs Online

135.1 Offline Planning (MDP)

- 通过计算得到所有量
- 已知完整模型
- 不需要真实执行

135.2 Online Planning (RL)

- 不能只靠计算
- 必须实际执行动作
- 从经验中学习

过程：

Act $\rightarrow$ Observe $\rightarrow$ Update

136 Model-Based Reinforcement Learning

136.1 Key Idea

两步：

- Step 1: 学习一个近似 MDP 模型
- Step 2: 解这个近似 MDP

136.2 Step 1: Learn the Model

数据形式：

$$s_0, a_1, r_1, s_1, a_2, r_2, s_2, \dots$$

估计转移概率：

$$\hat{T}(s, a, s') = \frac{\#(s, a, s')}{\#(s, a)}$$

奖励估计：

$$Reward(s, a, s') = r$$

其中：

- $\#(s, a, s')$ 表示出现次数
- $\#(s, a)$ 表示 (s, a) 出现次数

136.3 Step 2: Solve Learned MDP

例如使用 Value Iteration：

$$V(s) = \max_a \sum_{s'} \hat{T}(s, a, s') \left(Reward(s, a, s') + \gamma V(s') \right)$$

136.4 Advantages and Disadvantages

优点：

- 简单直观
- 只需计数和归一化
- 可以利用 MDP 算法

缺点：

- 需要维护所有 (s, a, s') 计数
- 状态空间大时非常昂贵

137 Model-Free Reinforcement Learning

不学习模型，直接学习值函数

137.1 Direct Evaluation

目标：

Given policy π , compute $V^\pi(s)$

基本思想：

- 按 π 执行

- 每次访问 s 记录一次回报
- 取平均

一次 sample:

$$sample_i(s) = R_1 + \gamma R_2 + \gamma^2 R_3 + \dots$$

估计:

$$V^\pi(s) = \frac{\sum_i sample_i(s)}{N(s)}$$

其中 $N(s)$ 是访问次数。

137.2 Direct Evaluation Problems

优点:

- 易理解
- 不需要 T 和 $Reward$

缺点:

- 每个状态独立学习
- 浪费状态之间的结构信息
- 收敛慢

138 Temporal Difference (TD) Learning

138.1 Big Idea

- 每次 transition 都更新
- 仍然是 policy evaluation
- 使用 running average

138.2 Sample Value

对于 transition $(s, \pi(s), s', r)$:

$$sample = r + \gamma V^\pi(s')$$

包含:

- Immediate reward
- Discounted future reward

138.3 TD Update Rule

$$V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha sample$$

等价写法:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha (sample - V^\pi(s))$$

其中:

$$0 \leq \alpha \leq 1$$

α 是学习率。

138.4 Temporal Difference

定义 TD error:

$$\delta = r + \gamma V^\pi(s') - V^\pi(s)$$

更新:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha \delta$$

This is a stochastic gradient update

138.5 Why TD is Better

- 不需要完整 episode
- 每一步都学习
- 收敛更快
- 样本效率更高

139 Problem with TD Value Learning

TD 只能做：

Policy Evaluation

若要改进策略：

$$\pi(s) = \arg \max_a Q(s, a)$$

但：

$$Q(s, a) = \sum_{s'} T(s, a, s') (Reward(s, a, s') + \gamma V(s'))$$

Transition model is unknown

因此：

Need to directly learn Q-values

140 Lecture 12 Summary

- RL = Online learning from experience
- Model-based RL: learn T and $Reward$, then solve MDP
- Model-free RL:
 - Direct Evaluation
 - Temporal Difference Learning
- TD uses bootstrap update
- To improve policy $\rightarrow$ learn Q directly

Next: Q-Learning

Lecture 13: Reinforcement Learning II

141 From Value Learning to Q-Learning

Recall:

TD value learning 只能做:

Policy Evaluation

若想改进策略:

$$\pi(s) = \arg \max_a Q(s, a)$$

因此:

We should directly learn Q-values

142 Recall: Value Iteration

$$V_{opt}(s) = \max_a \sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V_{opt}(s'))$$

143 Q-Value Iteration

定义:

$$Q_{opt}(s, a) = \sum_{s'} T(s, a, s') \left(R(s, a, s') + \gamma \max_{a'} Q_{opt}(s', a') \right)$$

区别:

- Value iteration: maximize over actions outside
- Q iteration: maximize inside successor state

问题:

What if T and R are unknown?

144 Temporal Difference Q-Learning

144.1 Big Idea

Q-Learning 是：

Sample-based Q-value iteration

给定一次 transition：

$$(s, a, s', r)$$

构造 sample：

$$sample = r + \gamma \max_{a'} Q(s', a')$$

更新规则：

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha sample$$

等价形式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

144.2 TD Error for Q

定义 TD error：

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

更新：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta$$

Off-policy learning

因为目标策略是 greedy，但行为策略可以探索。

145 Exploration vs Exploitation

常用策略：

ϵ -greedy

- 以概率 $1 - \epsilon$ 选择 $\arg \max_a Q(s, a)$
- 以概率 ϵ 随机选择动作

146 Generalizing Across States

Basic Q-learning:

Store a table of all $Q(s, a)$

问题:

- 状态太多
- 动作也很多
- 内存不可承受

因此:

Need function approximation

147 Feature-Based Representation

用特征向量表示状态:

$$f_1(s), f_2(s), \dots, f_n(s)$$

也可以表示 q-state:

$$f_i(s, a)$$

特征可以:

- 手工设计
- 神经网络自动学习

148 Linear Q-Function

定义线性函数:

$$Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

即:

$$Q(s, a) = \sum_i w_i f_i(s, a)$$

这是一个预测器。

149 Approximate Q-Learning

给定 transition (s, a, s', r) :

目标:

$$target = r + \gamma \max_{a'} Q(s', a')$$

误差:

$$difference = target - Q(s, a)$$

权重更新:

$$w_i \leftarrow w_i + \alpha difference f_i(s, a)$$

直观解释:

Adjust weights to reduce prediction error

150 Deep Q-Network (DQN)

使用神经网络近似:

$$Q(s, a; \theta) \approx Q^*(s, a)$$

输入:

$$s$$

输出:

$$Q(s, a_1), Q(s, a_2), \dots$$

一次 forward pass 得到所有动作的 Q 值。

151 Training the Q-Network

定义损失函数:

$$L(\theta) = \left(r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta) \right)^2$$

使用:

$$\nabla_{\theta} L(\theta)$$

进行 stochastic gradient descent。

152 Case Study: Atari Games

152.1 Problem Setup

- State: 原始像素
- Action: 游戏控制
- Reward: 得分

输入：

$$84 \times 84 \times 4$$

表示最近 4 帧。

输出维度等于动作数：

$$\dim = |A|$$

每个输出对应：

$$Q(s, a_i)$$

153 Lecture 13 Summary

- Q-learning 是 sample-based Q-value iteration
- 更新公式：

$$Q \leftarrow Q + \alpha(r + \gamma \max Q - Q)$$

- Off-policy learning
- Approximate Q-learning 使用特征
- DQN 使用深度神经网络
- Loss 是 TD error 的平方

Next: Advanced Deep RL or Policy Gradient

Lecture 14: Games I

154 Introduction to Game Problems

154.1 Game Playing Progress

历史里程碑:

- 1997: Deep Blue 击败 Kasparov
- 2016: AlphaGo 击败 Lee Sedol

说明:

Game search is extremely computationally expensive

155 Types of Games

155.1 General Games

- 多个 agent
- 每个 agent 有独立 utility
- 可能合作、竞争、无关

155.2 Zero-Sum Games

定义:

$$U_1(s) = -U_2(s)$$

特点:

- 一个最大化
- 一个最小化
- 纯对抗

Our focus: two-player zero-sum games

156 Two-Player Zero-Sum Game Model

定义一个游戏为：

- S_{start}
- $Actions(s)$
- $Succ(s, a)$
- $IsEnd(s)$
- $Utility(s)$
- $Player(s)$

例如 Chess：

$$Utility(s) = \begin{cases} +1 & \text{white wins} \\ 0 & \text{draw} \\ -1 & \text{black wins} \end{cases}$$

157 Game Tree

核心思想：

Game = Search Tree

每个节点是一个决策点。

每条 root-to-leaf 路径对应一个结果。

终止节点有已知 utility。

158 Value of a State

定义：

$$V(s) = \text{best achievable utility from } s$$

终止状态：

$$V(s) = Utility(s)$$

159 Adversarial Game Tree

若当前是 agent:

$$V(s) = \max_{s' \in \text{Successors}(s)} V(s')$$

若当前是 opponent:

$$V(s) = \min_{s' \in \text{Successors}(s)} V(s')$$

Assume opponent is optimal

160 Minimax Algorithm

定义 minimax value:

$$V(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ terminal} \\ \max_a V(\text{Succ}(s, a)) & \text{if MAX node} \\ \min_a V(\text{Succ}(s, a)) & \text{if MIN node} \end{cases}$$

策略:

$$\pi(s) = \arg \max_a V(\text{Succ}(s, a))$$

161 Minimax Complexity

若:

- branching factor = b
- max depth = m

时间复杂度:

$$O(b^m)$$

空间复杂度:

$$O(bm)$$

Chess:

$$b \approx 35$$

Go:

$$b \approx 250 \sim 300$$

Exact search is infeasible

162 Evaluation Function

为了解决深度过大问题：
使用 evaluation function。
定义：

$$Eval(s) \approx V_{minimax}(s)$$

用于：

- 非终止状态
- 近似估计

类比：

$Eval(s)$ 类似 A^* 中的 heuristic

163 Linear Evaluation Function

通常形式：

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s)$$

例如 chess 特征：

- material
- mobility
- king safety
- center control

164 Depth-Limited Minimax

基本思想：

- 限制最大深度 d_{max}
- 在 cutoff depth 使用 Eval(s)

更新规则保持：

$$V(s) = \begin{cases} Eval(s) & \text{if depth} = d_{max} \\ \max V(s') & \text{MAX node} \\ \min V(s') & \text{MIN node} \end{cases}$$

165 Depth-Limited Complexity

时间复杂度：

$$O(b^{d_{max}})$$

空间复杂度：

$$O(bd_{max})$$

但：

Only approximate decision

166 Lecture 14 Summary

- Game = adversarial search tree
- Zero-sum: one max, one min
- Minimax computes optimal strategy
- Complexity $O(b^m)$
- Evaluation function approximates value
- Depth-limited minimax makes search feasible
- Next: Alpha-Beta Pruning

Lecture 15: Exam Templates and Problem-Solving Patterns

Purpose: Provide structured answer templates for high-probability exam questions.

167 Template 1: Value Iteration (MDP Calculation)

Standard Question Form

Given:

- States S
- Actions A
- Transition $T(s, a, s')$
- Reward $R(s, a, s')$
- Discount γ
- Initial values $V^{(0)}(s)$

Perform one iteration of value iteration.

Answer Structure

Step 1: Write Bellman Update

$$V^{(t)}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{(t-1)}(s')]$$

Step 2: For each state:

- Compute value for each action separately
- Take maximum

Step 3: Write final updated values.

Exam Strategy: Always write the Bellman equation first.

168 Template 2: Q-Learning Update

Given transition (s, a, s', r) .

Formula

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Answer Structure

Step 1: Compute target

$$\text{Target} = r + \gamma \max_{a'} Q(s', a')$$

Step 2: Compute difference

$$\text{Difference} = \text{Target} - Q(s, a)$$

Step 3: Update

$$Q_{\text{new}} = Q + \alpha \times \text{Difference}$$

Memorize: Target $\rightarrow$ Difference $\rightarrow$ Update

169 Template 3: A* and Admissibility

Admissible Definition

$$h(n) \leq h^*(n)$$

where $h^*(n)$ is the true cost to goal.

Answer Pattern

Step 1: State definition.

Step 2: Compare $h(n)$ and $h^*(n)$.

Step 3: Conclude:

- If $h(n) \leq h^*(n)$ for all $n \rightarrow$ admissible.
- If overestimates $\rightarrow$ not admissible.

Write definition first to secure method marks.

170 Template 4: Minimax Calculation

Minimax Definition

$$V(s) = \begin{cases} Utility(s) & \text{terminal} \\ \max V(s') & \text{MAX node} \\ \min V(s') & \text{MIN node} \end{cases}$$

Answer Pattern

- Compute values from leaves upward.
- Compute all MIN nodes first.
- Then compute MAX nodes.
- Final root value is minimax value.

Always solve bottom-up.

171 Template 5: CSP Filtering

Forward Checking

Definition: Only checks neighbors of newly assigned variable.

Answer structure:

- Identify variables directly connected.
- Only those domains may change.

Arc Consistency

Definition: For each value in D_X , must exist consistent value in D_Y .

Propagation rule:

If D_X changes, recheck all arcs connected to X .

Forward checking = local. Arc consistency = may propagate globally.

172 Template 6: Backpropagation

General Strategy

Step 1: Forward pass.

- Compute intermediate variables.
- Compute final output.

Step 2: Backward pass.

Use chain rule:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}$$

Common Gates

Add gate: Gradient distributes equally.

Multiply gate: Gradients swap.

Sigmoid:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Write upstream gradient $\times$ local gradient explicitly.

173 Final Exam Strategy

- Always write the core equation first.
- Separate steps clearly.
- Compute action-by-action.
- Show intermediate values.
- Circle final answer.

Structure earns marks. Even if arithmetic has minor mistakes.

This section provides 10 high-probability exam problems with structured solutions and explanations.

174 Problem 1: BFS vs DFS vs UCS

Typical Question

Given a graph with edge costs. Write the path returned by BFS, DFS, and UCS.

Solution Strategy

BFS

- Expand shallowest node first.
- Uses FIFO queue.
- Ignores cost.

DFS

- Expand deepest node first.
- Uses stack.
- May not be optimal.

UCS

- Expand node with smallest $g(n)$.
- Uses priority queue.
- Optimal if step cost > 0 .

English Explanation: BFS optimizes depth. DFS optimizes memory. UCS optimizes cost.

175 Problem 2: A* Optimality

Core Formula

$$f(n) = g(n) + h(n)$$

Admissible Condition

$$h(n) \leq h^*(n)$$

Explanation:

If heuristic never overestimates, A^* is optimal.

中文解释:

只要启发函数不高估真实代价, A^* 一定能找到最优解。

176 Problem 3: Heuristic Dominance

Definition:

h_1 dominates h_2 if:

$$\forall n, h_1(n) \geq h_2(n)$$

and both are admissible.

Conclusion:

Larger admissible heuristic expands fewer nodes.

中文:

更大的 admissible heuristic 搜索更快。

177 Problem 4: CSP Forward Checking vs Arc Consistency

Forward Checking

Only checks neighbors of assigned variable.

局部传播。

Arc Consistency

If D_X changes, recheck all connected arcs.

可能全图传播。

Forward = local. Arc consistency = global propagation.

178 Problem 5: Hill Climbing vs Simulated Annealing

Hill Climbing:

- Always move to better neighbor.
- Gets stuck in local optimum.

Simulated Annealing:

Accept worse move with probability:

$$P = e^{\Delta E/T}$$

中文解释:

温度高时允许“犯错”，温度低时接近贪心。

179 Problem 6: Value Iteration Computation

Bellman Optimality Equation

$$V(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

Exam Tip:

Always:

- Compute action-by-action.
- Then take max.

中文:

先算每个 action，最后再取最大。

180 Problem 7: Policy Iteration

Two steps:

1. Policy Evaluation

$$V_{\pi}(s) = \sum_{s'} T(s, \pi(s), s') [R + \gamma V_{\pi}(s')]$$

2. Policy Improvement

$$\pi_{new}(s) = \arg \max_a Q(s, a)$$

中文解释:

固定策略算价值，再根据价值更新策略。

181 Problem 8: Q-Learning Update

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Interpretation:

New value = Old estimate + Learning rate $\times$ TD error.

中文:

旧估计 + 学习率 $\times$ 误差修正。

182 Problem 9: Backpropagation

Chain rule:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}$$

Multiply gate:

Gradient swaps.

Sigmoid:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

中文:

上游梯度 $\times$ 局部梯度。

183 Problem 10: Minimax and Depth-Limited Search

Minimax:

$$V(s) = \begin{cases} Utility(s) & \text{terminal} \\ \max V(s') & \text{MAX} \\ \min V(s') & \text{MIN} \end{cases}$$

Depth-limited:

Replace terminal utility with:

$$V(s) = Eval(s)$$

中文:

深度限制搜索，用评估函数代替真实终止值。

184 Final Advice

- Always write core formula first.
- Show intermediate steps.
- Structure answer clearly.
- Method marks are crucial.

Clear structure can earn partial credit even with arithmetic mistakes.

Lecture 16: Exam Operating System (Brain Freeze Safe Mode)

Purpose: Even if you forget all concepts under stress, this section allows you to mechanically execute answers. 即使在极度紧张下忘记所有概念，也可以机械执行答题步骤。

185 Part 1: Emergency Recognition Table

If the question contains these words, do NOT think. Just classify.

看到关键词时，不要思考，直接分类。

- "transition probability", "reward", "gamma"
→ MDP / Value Iteration
出现“转移概率”“奖励”“gamma” → 马尔可夫决策过程
- "Q table", "alpha", "update"
→ Q-learning
出现 Q 表、学习率 → Q-learning 更新
- "heuristic", "admissible", "optimal"
→ A* theory
出现启发函数、admissible → A* 理论题
- "domain reduction", "arc consistency"
→ CSP filtering
出现域缩减、弧一致性 → CSP
- "game tree", "MAX", "MIN"
→ Minimax
出现博弈树、MAX/MIN → Minimax
- "gradient", "partial derivative"
→ Backpropagation
出现梯度、偏导 → 反向传播

Rule: Keyword recognition replaces memory recall. 关键词匹配替代记忆。

186 Part 2: Forced First Line Rule

No thinking. Write the first equation immediately.

不要思考。第一行直接写核心公式。

MDP / Value Iteration

Write:

$$V(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

中文：状态价值等于对动作取最大，对下一个状态求期望。

If stuck: Compute action by action. Even partial calculation gives marks.

卡住时：逐个动作算。

Q-Learning

Write:

$$Q \leftarrow Q + \alpha [r + \gamma \max Q - Q]$$

Then compute:

1. Target 2. Difference 3. Update

中文步骤：

1. 算目标值 2. 算误差 3. 更新

A* Admissibility

Write:

$$h(n) \leq h^*(n)$$

Then:

Compare numbers. Conclude yes or no.

中文：

写定义，比大小，下结论。

Minimax

Write:

$$V(s) = \begin{cases} \max & \text{MAX node} \\ \min & \text{MIN node} \end{cases}$$

Then:

Compute from leaves upward.

中文:

从底向上算。

Backpropagation

Write:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}$$

Then:

Upstream gradient $\times$ local gradient.

中文:

上游梯度 $\times$ 局部梯度。

187 Part 3: Anti-Stuck Protocol

If brain freezes:

如果大脑空白:

1. Write the main equation.
先写核心公式。
2. Substitute numbers mechanically.
机械代数代入。
3. Do not re-derive theory.
不要推导理论。
4. Circle intermediate answers.
圈出中间结果。
5. Move on if time stuck.
卡住超过 2 分钟立即跳题。

188 Part 4: English Safety Notes

Common misunderstandings:

常见英文理解误区：

- "optimal" = 最优
- "admissible" = 不高估
- "consistent heuristic" = 单调性
- "policy evaluation" = 固定策略算价值
- "policy improvement" = 更新策略
- "depth-limited" = 有深度限制
- "terminal state" = 终止状态
- "utility" = 最终收益

If unsure of wording, focus on formula, not language.

如果英文看不懂，优先抓公式。

189 Part 5: Mechanical Exam Algorithm

Step 1: Classify by keyword.

Step 2: Write core formula.

Step 3: Substitute numbers.

Step 4: Structure answer clearly.

Step 5: Box final answer.

中文机械流程：

第一步分类第二步写公式第三步代数代入第四步分步骤写清第五步圈最终答案

190 Final Rule

Do not rely on memory. Rely on structure. 不要依赖记忆，依赖结构。